

Discovery Executive Summary

Project: discovery-14-July · Generated: 14/07/2026, 22:07:36

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 4 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—
2	Code Quality & Complexity Analysis	HIGH RISK	43 / 100 — Moderate
3	Frontend Modernization Analysis	HIGH RISK	—
4	Testing & Quality Assurance Analysis	HIGH RISK	—

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Service Layer (H2), Missing Repository Pattern (H3), Domain Boundary Violations (H8), Shared Database Coupling (H9), and Parallel API Runtimes (H10).

EXECUTIVE SUMMARY

Analysis covered **backend** (15 PHP application files: 6 API controllers, 4 Eloquent models, 2 services, 0 repositories), **frontend** (15 TS/TSX/JSX source files across pages, components, hooks, and store), and **dev-api** (6 JS files mirroring 18 Laravel routes) from `shende-shweta/FSDKC@main` via GitHub REST. The Klearcom monorepo runs three parallel runtimes with no repository layer and thin service coverage. Backend layering is the dominant risk: **25 controller-to-model Eloquent access points** bypass application services, **54 persistence calls** occur outside any repository abstraction (32 Eloquent + 22 MongoDB delegations), and **8 cross-domain query sites** in `DashboardController` couple Discovery and Connect models without an anti-corruption layer. Four of five MariaDB tables lack domain-exclusive ownership (**80% shared-table coupling**), and Laravel + `dev-api` duplicate reachability KPI math, IVR `buildTree`, and dashboard aggregation in parallel. Frontend architecture is comparatively healthy (avg **81 LOC** per view component,

centralized `api/client.ts`, max prop-drilling depth 1), but one legacy class component lacks lifecycle cleanup.
 Overall verdict: **High Risk**, driven by H2, H3, H8, H9, and H10.

1.1 Benchmark Ratings Summary

One row per hotspot. "Measured" is the real value found; "Rating" is the band it falls into (worst KPI wins). This table is the source for the Overall Codebase Rating banner above.

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	74 LOC avg	GOOD
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	25 access points	HIGH RISK
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	54 access points	HIGH RISK
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	1 (<code>LegacyDataMapper</code>)	MODERATE
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100% ORM	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	8	HIGH RISK
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	80% (4/5 tables)	HIGH RISK
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	81 LOC avg	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	7 files	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0	GOOD

F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	1 level	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	1 class component	MODERATE
H10	Parallel API Runtimes (additional)	Duplicate API implementations across runtimes	0	1	≥2	2 (Laravel + dev-api)	HIGH RISK

1.4 Actions Required

Hotspot	Action	Rating	Priority
H2 — Missing Service Layer	Introduce <code>ConnectMonitorService</code> , <code>DiscoveryJobService</code> , and <code>DashboardKpiService</code> ; move all 25 controller model-access points into application services with constructor DI.	HIGH RISK	CRITICAL
H3 — Missing Repository Pattern	Create repository interfaces for all 4 Eloquent models and 3 MongoDB collections; route 54 persistence access points through injected repository implementations.	HIGH RISK	CRITICAL
H5 — Shared Utility Abuse	Replace <code>LegacyDataMapper</code> <code>extract()</code> pattern with typed DTO mappers; remove <code>extract(\$filters)</code> from <code>LegacyReportController</code> .	MODERATE	MEDIUM
H8 — Domain Boundary Violations	Split cross-domain access in <code>DashboardController</code> and <code>LegacyReportController</code> ; enforce Discovery/Connect bounded contexts with published read APIs.	HIGH RISK	CRITICAL
H9 — Shared Database Coupling	Assign table ownership per domain in <code>docker/mariadb/init.sql</code> ; introduce per-domain migration files and integration views for cross-domain KPIs.	HIGH RISK	CRITICAL
H10 — Parallel API Runtimes	Deprecate <code>dev-api</code> duplicate runtime; consolidate to Laravel API with Docker Compose for local dev, or generate dev-api from OpenAPI spec.	HIGH RISK	CRITICAL
F5 — Legacy Component Patterns	Rewrite <code>LegacyMonitorPoller.jsx</code> as function component with <code>useEffect</code> cleanup; add Error Boundary around <code>LegacyDashboardWidget</code> .	MODERATE	MEDIUM

1.5 Expected Outcomes

- **Testable business logic:** Application services and repositories enable unit tests without HTTP or database bootstrapping, cutting test setup time for Connect reachability and Discovery tree workflows.
- **Independent domain evolution:** Bounded contexts with owned schemas let Discovery and Connect teams ship schema changes without cross-module regressions in dashboard KPIs.
- **Single source of truth:** Eliminating the parallel `dev-api` runtime removes behavioral drift between local development and production Laravel deployments.

- **Reduced change amplification:** Centralizing reachability calculation and IVR tree building in one service eliminates the current four-way duplication across controllers, services, and Node handlers.
- **Frontend stability:** Migrating the legacy class component and adding Error Boundaries prevents interval leaks and uncaught render errors during SPA navigation.

Deliverables saved:

- Full report: `target/docs/discovery/01-architecture-design.md` (577 lines — includes §1.2 evidence and §1.3 Mermaid diagrams)
- Pipeline summary: `agent-runs/20260714T195315_s84t1f/01-architecture-design-summary.md`

2. Code Quality & Complexity Analysis

OVERALL CODEBASE RATING — CODE QUALITY & COMPLEXITY

High Risk

Driven by High-Risk cyclomatic complexity in frontend page components (H1), oversized `ConnectPage` function (H3), cross-runtime business-logic duplication (H4), and parallel Laravel/dev-api workflow copies (H9).

EXECUTIVE SUMMARY

Analysis covered **backend** (13 PHP application files, 748 LOC), **frontend** (14 TS/TSX/JXS files, 874 LOC), and **dev-api** (6 JS files, 719 LOC) — **33 files / 2,341 LOC** total — from `shende-shweta/FSDKC@main` via GitHub REST and raw content fetch. No cyclomatic-complexity linter (`eslint-plugin-complexity`, `phpmd`, Sonar) is configured; metrics were derived by manual branch/loop counting. The dominant risks are **frontend page-level complexity** (`ConnectPage.tsx` cyclomatic complexity **35**, **201 LOC** single component) and **cross-runtime business-logic duplication** (~12.4% of codebase duplicated between Laravel and `dev-api` for KPI aggregation, realtime test orchestration, and IVR `buildTree`). No files exceed 1,000 LOC (largest: `dev-api/src/server.js` at 224 LOC). Git history is shallow (**3 commits**, single author `ksabai-gl` since June 2026), so churn and ownership signals are healthy but low-confidence. Overall verdict: **High Risk**, driven by H1, H3, H4, and H9.

2.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	High Cyclomatic Complexity	Max complexity per method	<10	10–20	>20	35 (<code>ConnectPage</code>)	HIGH RISK

H2	Large Classes	Largest class LOC	<300	300–1000	>1000	224 LOC (<code>dev-api/src/server.js</code>)	GOOD
H3	Large Functions	Largest function LOC	<50	50–200	>200	201 LOC (<code>ConnectPage</code>)	HIGH RISK
H4	Business Logic Duplication	Duplicated business logic %	<5%	5–10%	>10%	12.4% (290 / 2,341 LOC)	HIGH RISK
H5	Duplicate Code (general)	Overall duplicate code %	<5%	5–10%	>10%	7.7% (180 / 2,341 LOC)	MODERATE
H6	High Churn Areas	Monthly changes (top files)	<5	5–10	>10	2 changes/mo (top app files)	GOOD
H7	Defect-Prone Files	Fix commits (hottest file)	1–3	4–5	>5	1 fix commit (<code>ConnectController.php</code>)	GOOD
H8	Ownership Issues	Top-author ownership %	>80%	60–80%	<60%	100% (1 author / 3 commits)	GOOD
H9	Parallel Runtime Duplication (additional)	Duplicated workflow LOC across Laravel + dev-api / backend LOC	<5%	5–15%	>15%	19.8% (290 / 1,467 backend+dev-api LOC)	HIGH RISK
H10	God Page Components (additional)	Largest page component LOC (UI + data + realtime)	<150	150–300	>300	208 LOC (<code>ConnectPage.tsx</code>)	MODERATE

Hotspot Score breakdown

Component	Weight	Sub-score (0–100)	Weighted
Cyclomatic Complexity	25%	72	18.0
Code Churn	25%	12	3.0
Defect Density	20%	18	3.6
Class/Function Size	15%	68	10.2
Business Logic Duplication	10%	78	7.8

Developer Ownership Risk	5%	8	0.4
Hotspot Score	100%		43 / 100

2.5 Actions Required

Hotspot	Action	Rating	Priority
H1 — High Cyclomatic Complexity	Extract <code>useModulePageQueries</code> hook; split <code>ConnectPage</code> and <code>DiscoveryPage</code> into ≤80 LOC sub-components; enable ESLint complexity rule (max 15) on <code>frontend/src/pages/</code> .	HIGH RISK	CRITICAL
H3 — Large Functions	Decompose <code>ConnectPage</code> (201 LOC) into <code>useConnectPageState.ts</code> + 3 presentational components; apply same pattern to <code>DiscoveryPage</code> (154 LOC).	HIGH RISK	HIGH
H4 — Business Logic Duplication	Create <code>DashboardKpiCalculator</code> and shared test-step definitions in Laravel; remove duplicated KPI and realtime blocks from <code>dev-api/src/server.js</code> and <code>dev-api/src/realtime.js</code> .	HIGH RISK	CRITICAL
H5 — Duplicate Code	Extract <code>TreeBuilder</code> utility from three <code>buildTree</code> copies; add <code>jscpd</code> / <code>phpcpd</code> CI gate at 5% threshold.	MODERATE	MEDIUM
H9 — Parallel Runtime Duplication	Deprecate <code>dev-api</code> runtime or auto-generate from Laravel OpenAPI; add contract tests until removal.	HIGH RISK	CRITICAL
H10 — God Page Components	Introduce <code>ModulePageLayout</code> , <code>EntityForm</code> , and <code>TestSessionPanel</code> shared components; cap page files at 120 LOC.	MODERATE	MEDIUM

2.6 Expected Outcomes

- **Lower defect rate on UI changes:** Splitting `ConnectPage` / `DiscoveryPage` and extracting shared hooks reduces cyclomatic complexity from 35 to a testable target of <15 per function.
- **Single source of truth for KPIs and test pipelines:** Consolidating dashboard and realtime logic in Laravel eliminates the ~12% duplicated business-rule surface between production and `dev-api`.
- **Safer refactors:** Extracting `TreeBuilder` and domain services lets IVR tree and reachability changes propagate from one module instead of three `buildTree` copies.
- **Faster reviews:** Smaller page components and a complexity lint gate keep new features from re-expanding god components.
- **Clearer ownership signals:** As contributor count grows, shallow-history baselines will be replaced by meaningful churn metrics already instrumented in this report template.

Full report saved: `target/docs/discovery/02-code-quality-complexity.md`

Pipeline summary: `agent-runs/20260714T214921_12dnvv/02-code-quality-complexity-summary.md`

3. Frontend Modernization Analysis

OVERALL CODEBASE RATING — FRONTEND MODERNIZATION

High Risk

Driven by High-Risk UI Component Duplication (H1), Global State Dependencies (H4), Inline Styles Without Design Tokens (H6), and Missing Accessibility Roles (H7).

EXECUTIVE SUMMARY

The Klearcom frontend is a compact React 19 / Vite 6 / TypeScript SPA with 14 source files (8 view components across `pages/` and `components/`). The stack is largely modern — hooks, TanStack React Query, Zustand, and React Router dominate data flow — but two domain pages (`ConnectPage.tsx`, `DiscoveryPage.tsx`) duplicate a shared workbench shell (entity form, live feed, two-column grid, transcript panel), inflating UI duplication to 25%. A legacy class component (`LegacyMonitorPoller.jsx`) lacks `componentWillUnmount` cleanup and leaks intervals, and two orphan legacy modules (`LegacyMonitorPoller`, `LegacyDashboardWidget`) coexist with the React Query stack. Six of eight view components read shared global state (Zustand store, React Query cache, or the singleton `api` client), 30 inline `style={{}}` occurrences bypass the CSS token system in `index.css`, and zero `aria-*` / `role` attributes were found across all view components. Overall verdict: **High Risk**, driven by H1 (UI duplication), H4 (global state coupling), H6 (inline styles), and H7 (missing accessibility).

3.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	UI Component Duplication	Duplicate components %	<5%	5–10%	>10%	25.0% (2/8)	HIGH RISK
H2	Legacy Class-Based Components	Modern component adoption %	>90%	70–90%	<70%	87.5% (7/8)	MODERATE
H3	Massive Components	Largest component LOC	<200	200–500	>500	222 (<code>ConnectPage.tsx</code>)	MODERATE
H4	Global State Dependencies	Components reading global state %	<30%	30–60%	>60%	75.0% (6/8)	HIGH RISK
H5	Complex State Management	Max prop-drilling depth	<3	3–5	>5	1 level	GOOD

H6	Inline Styles / No Design Tokens (additional)	Total <code>style={{}}</code> occurrences (target <10)	<10	10–20	>20	30 across 7 files	HIGH RISK
H7	Missing Accessibility Roles (additional)	Components with <code>aria-*</code> or <code>role</code> (target >80%)	>80%	50–80%	<50%	0% (0/8)	HIGH RISK

3.5 Actions Required

Hotspot	Action	Rating	Priority
H1 — UI Component Duplication	Extract <code>DomainWorkbenchPage</code> , <code>TranscriptPanel</code> , and <code>EntityFormCard</code> from <code>ConnectPage.tsx</code> and <code>DiscoveryPage.tsx</code> ; reduce duplicate shell to domain hooks only.	HIGH RISK	CRITICAL
H2 — Legacy Class-Based Components	Convert <code>LegacyMonitorPoller.jsx</code> to <code>useMonitorReachability</code> hook; delete or migrate <code>LegacyDashboardWidget.tsx</code> to React Query.	MODERATE	MEDIUM
H3 — Massive Components	Split <code>ConnectPage.tsx</code> (222 LOC) and <code>DiscoveryPage.tsx</code> (176 LOC) into container hooks + presentational sub-components; target <150 LOC per page.	MODERATE	MEDIUM
H4 — Global State Dependencies	Replace <code>useUiStore</code> selection with URL search params; add <code>frontend/src/api/queryKeys.ts</code> ; scope query invalidation to domain keys.	HIGH RISK	HIGH
H6 — Inline Styles	Add CSS utility classes to <code>index.css</code> ; migrate 30 inline <code>style={{}}</code> occurrences (18 in domain pages) to token-based classes.	HIGH RISK	HIGH
H7 — Missing Accessibility	Add <code>aria-live</code> to <code>LiveTestFeed</code> , keyboard-accessible row selection in domain tables, and <code>htmlFor</code> on form labels.	HIGH RISK	HIGH

3.6 Expected Outcomes

- **Shared component library** eliminates the 25% duplicate page-shell pattern; Connect and Discovery fixes ship once via `DomainWorkbenchPage` and `TranscriptPanel`.
- **Hooks-first migration** of `LegacyMonitorPoller` removes interval leaks and aligns all data access on React Query with testable custom hooks.
- **Scoped state** (URL params + domain query keys) reduces cross-feature invalidation fan-out and makes monitor/job selection shareable via deep links.
- **Design-token consistency** from replacing 30 inline styles lowers theming cost and aligns with the existing `:root` token system in `index.css`.
- **Accessibility baseline** (`aria-live`, keyboard rows, label associations) makes realtime test feeds and data tables operable for keyboard and screen-reader users in an operations-focused product.

Full report saved to `target/docs/discovery/03-frontend-modernization.md` (orchestration UI will convert to PDF). Pipeline summary: `agent-runs/20260714T220150_tw7uq/03-frontend-modernization-summary.md` . Analysis source: `shende-shweta/FSDKC@main` (14 TS/TSX/JSX files, 8 view components).

4. Testing & Quality Assurance Analysis

OVERALL CODEBASE RATING — TESTING & QUALITY ASSURANCE

High Risk

Driven by High-Risk untested critical logic (H1), low coverage (H2), missing integration tests (H3), missing contract tests (H4), absent E2E tests (H7), and assertion-free unit tests (H8).

EXECUTIVE SUMMARY

Test-suite health across the FSDKC monorepo is **critically thin**. The backend ships **PHPUnit 11** with only **2 unit test files** (3 test methods) that assert hard-coded arrays and random numbers — none import `App\` production classes. The React 19 / Vite 6 / TypeScript frontend (**15 source files**) and Node `dev-api` (**6 source files**) have **zero** test files and no Jest, Vitest, Cypress, or Playwright configuration. Estimated overall coverage by test-file-to-source ratio is **~6%** (2 test files / 36 application source files); per-layer split is **backend ~13% file ratio (0% effective), frontend 0%, and dev-api 0%**. No coverage reports (`lcov`, `clover.xml`) exist in the repository. GitHub Actions runs `vendor/bin/phpunit` against MariaDB on every PR (backend gate), but the frontend job only runs `npm run build` and `dev-api` is excluded entirely. The highest-risk gaps are untested Discovery/Connect orchestration (`RealTimeTestService`, six API controllers), no integration or contract tests for **19 public API routes**, and no E2E coverage for realtime SSE workflows. Overall verdict: **High Risk**.

5.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Untested Critical Logic	Critical modules with zero tests	0	1–3	>3	9 modules	HIGH RISK
H2	Low Test Coverage	Overall coverage %	>80%	50–80%	<50%	~6% estimated (2/36 app source files; no coverage report)	HIGH RISK
H3	Missing Integration Tests	Boundaries covered %	>70%	30–70%	<30%	0% (0 Feature/HTTP/DB tests)	HIGH RISK

H4	Missing Contract Tests	APIs with contract tests %	>80%	40–80%	<40%	0% (0/19 endpoints)	HIGH RISK
H5	Flaky / Skipped Tests	Skipped/flaky test count	0	1–5	>5	0	GOOD
H6	No CI Test Gate	Tests enforced in CI	Required gate	Runs, not required	No CI test run	Backend PHPUnit on PR; frontend build-only; dev-api excluded	MODERATE
H7	No End-to-End Tests (additional)	Critical user flows with E2E specs	≥2 flows	1 flow	0 flows	0 E2E specs	HIGH RISK
H8	Assertion-Free Unit Tests (additional)	Unit tests exercising production code %	>80%	40–80%	<40%	0% (0/2 tests import <code>App\</code> code)	HIGH RISK

5.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — Untested Critical Logic	Add PHPUnit tests for <code>RealTimeTestService</code> , all six API controllers, and <code>MongoService</code> ; add Vitest tests for <code>useRealtimeTest</code> , pages, and <code>api/client.ts</code> ; add or remove dev-api with parity tests.	HIGH RISK	CRITICAL
H2 — Low Test Coverage	Establish coverage baselines (PHPUnit <code>--coverage-text</code> , Vitest <code>--coverage</code>); enforce 75% backend / 60% frontend thresholds in CI before refactors.	HIGH RISK	CRITICAL
H3 — Missing Integration Tests	Create <code>backend/tests/Feature/</code> with <code>RefreshDatabase</code> + MariaDB; test Discovery job lifecycle, Connect check flow, and SSE streaming boundaries.	HIGH RISK	HIGH
H4 — Missing Contract Tests	Publish OpenAPI spec; add JSON Schema contract tests for all 19 API endpoints; validate TypeScript types against fixtures.	HIGH RISK	HIGH
H6 — No CI Test Gate	Add <code>vitest run</code> to frontend CI job; add dev-api test job or deprecate runtime; mark all test jobs as required PR checks.	MODERATE	MEDIUM
H7 — No End-to-End Tests	Add Playwright with Docker Compose; cover Discovery create → start → stream and Connect create → check → alert flows.	HIGH RISK	HIGH
H8 — Assertion-Free Unit Tests	Replace <code>HealthTest</code> and <code>ReachabilityCalculationTest</code> with tests that import <code>App\</code> classes; fail CI on assertion-free unit tests.	HIGH RISK	MEDIUM

5.5 Expected Outcomes

- **Critical paths protected:** Discovery job orchestration, Connect reachability calculation, and MongoDB event streaming have automated regression tests before any service-layer extraction.
 - **CI catches full-stack regressions:** Required PHPUnit + Vitest + Playwright gates on every PR prevent untested changes from merging.
 - **Contract stability:** JSON Schema tests for 19 API endpoints prevent breaking changes to `{ data: ... }` envelopes and SSE event shapes consumed by the React SPA.
 - **Effective coverage above 75%:** Replacing smoke tests and adding Feature/Vitest suites raises measured coverage from ~6% to refactor-safe levels.
 - **Parallel runtime drift eliminated:** Contract parity tests (or dev-api removal) ensure local development matches production Laravel behavior.
-

Deliverables saved:

- Full report: `target/docs/discovery/05-testing-and-quality-assurance.md`
- Pipeline summary: `agent-runs/20260714T220150_tw7uq/05-testing-and-quality-assurance-summary.md`

Analysis covered **backend** (15 PHP files, 2 PHPUnit tests), **frontend** (15 TS/TSX/JSX files, 0 tests), and **dev-api** (6 JS files, 0 tests) from `shende-shweta/FSDKC@main` via GitHub REST API.